Documentation IA Zappy : Stratégie, Évolution et Architecture

1. Vision Globale et Philosophie

L'objectif de notre client IA pour le projet Zappy repose sur une architecture d'Intelligence en Essaim. Chaque agent s'exécute de manière asynchrone dans un processus distinct, sans mémoire partagée. La survie, la collecte et l'élévation dépendent d'une communication réseau chiffrée et d'une machine à états robuste, totalement dictée par les événements du serveur.

2. L'Évolution de l'Architecture : Ambitions, Échecs et Pragmatisme

La conception de cette IA est passée par plusieurs itérations. Nous avons dû revoir nos objectifs initiaux, très ambitieux, pour nous adapter aux contraintes réelles du serveur et assurer une stabilité maximale.

Phase 1 : L'approche naïve et le piège temporel (La Fork Bomb)

- **L'idée initiale :** Utiliser des chronomètres internes (time.time()) et des pauses (time.sleep()) pour gérer l'éclosion des œufs après un Fork.
- **L'échec :** La fréquence variable du serveur (-f) rendait nos timers obsolètes. La Reine, pensant que les œufs n'éclosaient pas, spammait la commande de ponte, créant une "Fork Bomb" qui faisait crasher le système.
- **La solution :** Le passage à l'Événementiel Pur. La gestion de la population s'est basée sur les variations de place entre deux envois de Connect_nbr, rendant l'IA agnostique au temps.

Phase 2 : L'ambition démesurée de "La Grande Muraille"

- **L'idée initiale :** Mettre en place un système de 4 castes ultra-spécialisées dès le niveau 3 :
  - *La Reine :* Aspirateur de pierres et lanceuse de rituels.
  - *La Garde Royale :* Fournisseurs exclusifs de pierres pour la Reine.
  - *Les Protecteurs :* Une phalange formée en croix autour du rituel spammant la commande Eject.
  - *Les Ouvriers :* Des farmeurs de nourriture dédiés au ravitaillement.
- **L'échec :** Bien que conceptuellement brillante, cette stratégie de micro-management s'est heurtée à la réalité asynchrone de Zappy. La synchronisation parfaite requise pour la Phalange et les transferts d'inventaire causait des blocages logiques et une surdité sélective aux messages radio.

Phase 3 : Le Pragmatisme (La version actuelle)

- **La décision :** Revoir l'architecture à la baisse pour maximiser la résilience. Nous avons supprimé les castes superflues (Guard, Protector) pour nous concentrer sur une binarité efficace : **QUEEN** et **PEASANT**, couplée à une machine à états très détaillée.

3. Architecture Logique Actuelle

3.1. Le Système de Rôles Simplifié

Rôle	Description et Comportement
QUEEN (Reine)	Prend l'ID exclusif 9999. Elle est le chef d'orchestre de l'essaim. Elle gère la croissance démographique via les Fork et lance les processus de clones. Elle centralise le regroupement pour lancer les élévations multiples.
PEASANT (Paysan)	Le rôle par défaut. Ils farment pour survivre et atteindre le niveau 3. S'ils ont plus de 25 unités de nourriture, ils répondent à l'appel de la Reine pour le regroupement. S'ils détectent une famine chez leurs alliés, ils peuvent basculer en mode FEEDER.

3.2. La Machine à États (State Machine)

Chaque cycle de boucle met à jour l'état interne de l'agent. Ses actions sont définies par une énumération stricte :

- **BEGINNER :** Phase de démarrage pour les niveaux 1 et 2, objectif d'élévation rapide.
- **SURVIVAL :** Si la nourriture tombe sous un seuil critique, l'agent abandonne toute autre tâche pour chercher de la nourriture.
- **FARMING :** Recherche active de pierres spécifiques dictées par les besoins du niveau supérieur ou de la Reine.
- **GROUPING :** Mouvement vers un allié ou la Reine (via le k du broadcast) pour préparer un rituel.
- **ELEVATION :** Nettoyage de la tuile et lancement ou attente de l'Incantation.
- **FEEDER / WAIT :** États de support logistique où l'agent dépose ses surplus ou se met en veille stratégique.

4. Architecture du Code et Fonctions Majeures

L'aspect technique du projet repose sur une structuration modulaire rigoureuse en Python, séparant la gestion réseau, la boucle cognitive de l'agent, et ses sous-modules spécialisés.

4.1. Core System (Le Cœur du Système)

- **main.py** : Point d'entrée. Exécute le handshake de départ (wait_for_welcome, wait_for_infos), puis instancie et lance l'agent.
- **server_link.py (ZappyNetwork)** : Gère la connexion réseau TCP avec un socket **non-bloquant** (self.sock.setblocking(False)). Sa méthode receive_messages() lit les données en continu sans interrompre le cycle de l'agent, garantissant la nature asynchrone de l'IA.
- **command.py (ZappyCommand)** : Abstraction de l'API Zappy. Maintient deux files d'attente distinctes (system_queue et broadcast_queue). Des méthodes dédiées comme _parse_look() et _parse_inventory() formatent directement les chaînes textuelles du serveur en listes et dictionnaires exploitables.
- **Ai_agent.py (ZappyAgent)** : Le centre névralgique de l'IA. Sa boucle infinie run() articule la vie de l'agent :
    1. update_internal_state() met à jour vision et inventaire.
    2. process_radio_and_demographics() traite l'historique asynchrone du réseau.
    3. evaluate_situation() prend les décisions d'état de haut niveau.
    4. act_based_on_role() exécute l'action finale (via la machine à états).

4.2. Les Sous-Modules (Le Cerveau et ses Facultés)

- **agent_role.py (RoleTask)** : Régit les transitions globales selon la caste de l'agent (ex: act_queen(), act_peasant()).
- **agent_state.py (StateComportment)** : Exécute les algorithmes métiers de bas niveau pour un état donné (ex: act_farming(), act_elevating()). C'est ici qu'intervient la logique comme _clean_and_set_stones() pour nettoyer la tuile avant un rituel.
- **farm.py (Farmer)** : Gère l'économie du jeu. Grâce à get_missing_stones() et get_missing_stones_for_max_level(), ce module calcule les écarts entre l'inventaire actuel (ou global) et la "liste de courses" nécessaire pour une élévation locale ou le méga-rituel de la Reine.
- **pathfinding.py (Pathfinder)** : Pilote les déplacements spatiaux. Sa méthode search_and_take_items() s'appuie sur un modèle de scoring (_score_tile()) qui pondère la densité d'items d'une tuile par le nombre d'actions requises pour l'atteindre (Forward/Left/Right).
- **grouping.py (Grouper)** : Traduit les appels de ralliement. Convertit la valeur de la direction sonore k (1 à 8) en séries d'instructions de mouvement (_move_towards_sound()).
- **stealth_comms.py (StealthComms)** : Le chiffrement. Assure que chaque commande Broadcast est munie d'un token asymétrique (_generate_stealth_token()) et validé à la réception (is_token_valid()) pour protéger l'essaim de l'espionnage et de l'usurpation ennemie.